SRUJANA TECH FEST 2026 · FINANCE THEME · PROBLEM STATEMENT F-03

ClearRun

Predictive cash-flow management for small Indian businesses: see a cash gap coming, get paid faster, and give credit wisely.

Team	OUTLIER
Live app	https://clearrun-app3.vercel.app
Source code	https://github.com/rohinii-dev/clearrun
Report date	26 October 2026 (version 5)

1. Project overview
2. Problem understanding
3. Proposed solution
4. Innovation and differentiation
5. System architecture
6. Implementation details
7. Achievements during the hackathon
8. Testing and validation
9. AI, data and external dependencies
10. User interface and demonstration
11. Limitations and incomplete work
12. Real-world viability and impact
13. Future scope
14. Setup and reproducibility
15. Team contributions and references

1 Project overview

Project title	ClearRun: cash flow planning for small businesses
Team name	OUTLIER
Theme	Finance
Problem statement	F-03: Predictive cash-flow management for small businesses

Executive summary. Small B2B businesses often run short of cash even when they are profitable. Customers pay weeks late, but salaries, rent and suppliers must be paid on time. ClearRun is a web app that gives the owner:

- a 13-week cash forecast with what-if scenarios, and a 1,500-run simulation of the *chance* of falling below a safe reserve;
- invoice capture by photo or PDF, with every value the AI reads checked by fixed rules;
- an automatic payment reminder ladder (gentle, then formal, then legal notice) that uses Indian MSMED Act provisions;
- a customer credit-risk score, built from each customer's real payment behaviour, with concrete advice for the next deal;
- shared team workspaces and Google sign-in.

The app is deployed and working online. Section 11 states clearly what is not yet proven.

2 Problem understanding

Target users

Owners and accountants of small B2B businesses in India, such as manufacturers, distributors and traders, that sell on credit terms of 15 to 60 days.

Pain points

Cash is tracked in someone's head or a spreadsheet. A cash gap is noticed only when it arrives. Late payers are chased by hand, invoices are typed in manually, and credit is given by gut feel.

Why it matters. A business can show a profit on paper while having no cash on hand. A business that cannot pay salaries or suppliers on time loses trust, discounts and sometimes the business itself. Delayed payment to micro and small enterprises is a big enough problem in India that it has its own law, the MSMED Act, 2006 (Sections 15 to 18). That law entitles registered MSEs to compound interest on late payments and a dispute route through the MSE Facilitation Council.

Assumptions. Users can state their current balance and expected receipts and payments. Most of their customers are businesses with email addresses. Some users are Udyam-registered MSEs. Users have little payment history at first, so the models must work with small amounts of data.

Specific needs addressed: (1) early warning of a shortfall, (2) faster collection, (3) safer credit decisions, (4) less manual data entry, (5) a shared view for the team.

3 Proposed solution

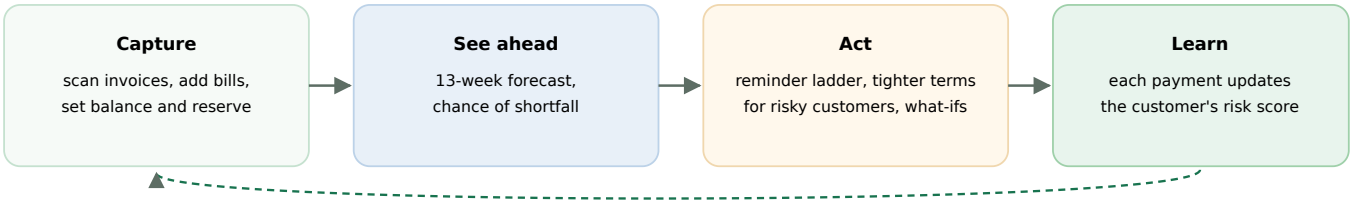


Figure 1. The loop ClearRun supports: capture, see ahead, act, learn.

Feature	How it works
13-week forecast and scenarios	Opening cash plus expected receipts minus planned payments, week by week, compared with the owner's reserve. Sliders delay the next customer payment, add a sudden expense, or change sales.
Chance of a shortfall	A Monte Carlo simulation of 1,500 possible 13-week futures, with realistic variation in when receipts arrive. It reports the chance of going below the reserve and of negative cash.
Invoices and scanning	The user photographs or uploads an invoice. Gemini reads it, and fixed rules check the total against the amount in words, the subtotal plus GST against the total, and the GSTIN checksum. Uncertain fields are highlighted. Paid invoices build payment history.
Reminder ladder	Gentle reminders 7 days before and on the due date, formal reminders at 7 and 15 days overdue, and a legal notice at 30 days. The legal notice needs the owner's approval for each invoice and can also be printed. A daily job sends each step at most once.
Customer risk and deal check	A 0–100 score from six payment signals. The classes are Full credit, Limited credit and Advance only. Suggested terms, advance and credit limit, and the realistic date the money will arrive, can be added to the forecast in one click.
Teams and sign-in	Google or phone and password sign-in. Invite codes let teammates share a workspace, and saving merges changes so nobody's work is lost.

User journey. Sign in → enter the cash balance and reserve → scan or type invoices and bills → read the forecast and the chance of a shortfall → try scenarios → turn on reminders for overdue invoices → check customer risk before the next big order → mark invoices received as money arrives.

4 Innovation and differentiation

- **Probability instead of a single line.** Most simple cash tools draw one forecast line. ClearRun also shows the *chance* of a shortfall, which is what an owner needs to decide how worried to be.
- **AI only where it adds value, with checks.** Only invoice reading uses AI. The model copies values exactly as printed, and our own code converts and cross-checks them. A failed check triggers a second, targeted reading, and fields are highlighted by confidence. A misread total is caught before it reaches the forecast.
- **Explainable credit scores.** Every risk score lists its reasons, such as "usually pays 50 days late" or "broke 5 of 7 promises". The advice follows from those reasons: a dated cheque for promise-breakers, split delivery for unpredictable payers. New customers are compared with the most similar existing customers, and that comparison is shown too.
- **Built for India.** Amounts in lakh and crore, day-first dates, GSTIN validation, MSMED Act Section 16 interest estimates, and Section 18 wording in the legal notice for Udyam-registered sellers.
- **From warning to action.** The forecast, reminders and risk advice feed each other. A deal check can be added to the forecast in one click, and each payment received becomes history for the next score.

5 System architecture

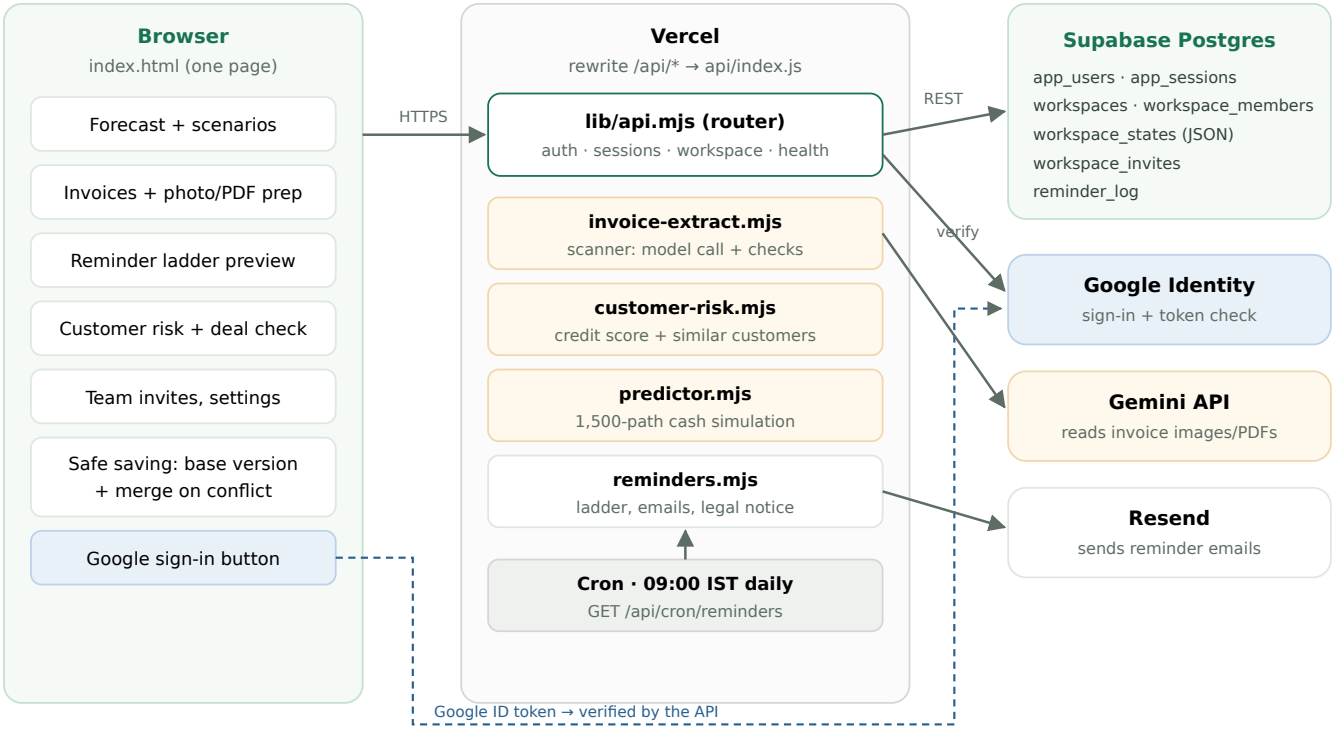


Figure 2. Components and connections. The amber boxes are the three models.

Layer	Technology and role
Front end	One page (`index.html`) in plain HTML, CSS and JavaScript, with seven views and a demo mode. It prepares photos in the browser (rotating, resizing and brightening them) before upload.
Back end	Node.js 20 API (`lib/api.mjs`). It runs on Vercel as one serverless function (`api/index.js` , with every `/api/*` path rewritten to it). Locally it runs through `server.mjs` .
Models	`invoice-extract.mjs` (scanner), `customer-risk.mjs` (credit score), `predictor.mjs` (Monte Carlo simulation), and `reminders.mjs` (ladder, emails and legal notice).
Database	Supabase Postgres, reached through its REST interface with a server-only key. Row-level security is on, and the anonymous and signed-in roles have no rights.
External services	Google Identity (sign-in), Gemini API (invoice reading), Resend (email), and Vercel Cron (09:00 IST daily job).

Database design

Table	Purpose and key columns
`app_users`	id, username (unique ignoring case), phone (unique), email, google_sub (unique), password_hash (scrypt), workspace_id
`app_sessions`	token_hash (SHA-256 of the cookie token), user_id, expires_at
`workspaces` , `workspace_members`	One workspace per business. Members have the role owner or member.
`workspace_states`	workspace_id, **state** (JSON: cash, cash events, invoices, reminder settings, customer profiles, scenario values), updated_at (used for safe saving)
`workspace_invites`	code_hash, workspace_id, expires_at (7 days)
`reminder_log`	workspace_id, invoice_id, stage, status, sent_at. It allows only one row per (workspace, invoice, stage), so no email is sent twice.

API

Endpoint	Purpose
`POST /api/auth/signup` · `/signin` · `/google`	Create an account, sign in, or sign in with Google (the token is verified on the server)
`GET/DELETE /api/session`	Restore the session or sign out
`GET/PUT /api/workspace`	Load or save. A stale save gets 409 and the browser merges both sets of changes.
`POST /api/workspace/invite` · `/join`	Create or use a team invite code
`POST /api/invoice/extract`	Photo or PDF in, checked fields with confidence out
`GET/PUT/POST /api/risk`	Customer profiles, risk overview, and deal assessment
`POST /api/predict`	Monte Carlo shortfall probabilities
`POST /api/reminders/preview` · `/test` · `GET /list`	Ladder preview, a test email to the owner, and reminder history
`GET /api/cron/reminders`	The daily job. It needs a secret that only Vercel sends.
`GET /api/health` · `/api/config`	The setup check (which settings are present, whether the database can be reached), and the Google client ID for the browser

6 Implementation details

Invoice scanner

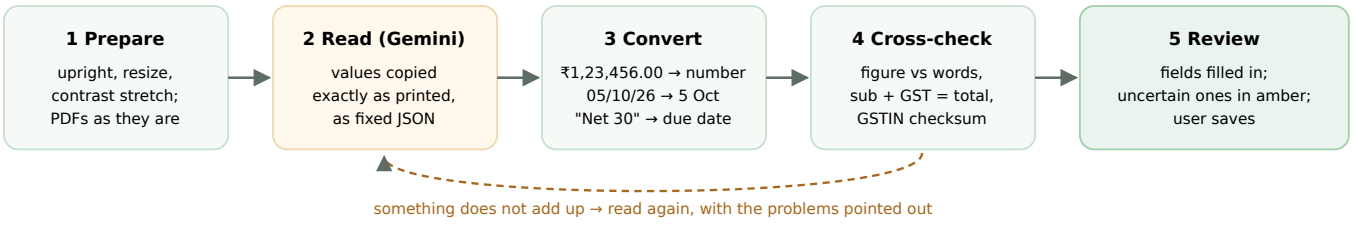


Figure 3. Scanner pipeline. Only step 2 uses AI. Steps 3 and 4 are fixed rules that always give the same result.

The prompt tells Gemini to copy values exactly as printed: `totalAmountText`, `amountInWords`, `invoiceDateText`, and the rest. Our code then converts amounts (₹1,23,456.00, Rs. 45,000/-), turns amounts in words into numbers (including lakh, crore and paise), and reads dates day-first, flagging any that could be read either way. It also works out a due date from payment terms such as "Net 30". The checks are: total against amount in words; subtotal + GST + round-off = total; the GSTIN mod-36 checksum; the buyer and seller not being the same; and sensible dates. If any check fails, the invoice is read again with the problems listed, and the reading that passes more checks is kept. To stay available, the scanner tries Gemini 3.8 Flash, then 3.6 Flash, then 3.5 Flash-Lite. Each attempt gets at most 18 seconds, and the whole scan finishes within about 50 seconds, under Vercel's 60-second limit.

Customer risk model

This is a JavaScript port of the team's Python model, with identical rules. It uses six signals: lateness (median days late, up to 25 points), unpredictability (80th percentile minus median, up to 15), very late share (45+ days, up to 20), broken promises (up to 15), overdue now (oldest invoice past due, up to 15), and slowing down (10). Scores of 0–30 mean Full credit, 31–55 Limited credit, and 56+ Advance only. New customers are compared with the 5 most similar customers that have at least 5 paid invoices:

```
distance = 3.0·[type differs] + 1.5·|ln(1+years)Δ|/ln31 + 1.0·|ln(order)Δ|/ln100 +
1.0·|termsΔ|/60
weight = 1/(distance+0.1)        own-history weight = n/(n+5) below 8 paid invoices
credit limit = factor × monthly billing × max(1, (credit days + typical delay)/30),  factor
1.5 / 1.0 / 0.5
```

Payment history is built automatically from ClearRun invoices (paid date minus due date, and overdue days on open invoices), alongside any history typed in by hand.

Cash shortfall simulator

The simulator runs 1,500 versions of the 13 weeks. Each expected receipt lands in its planned week, shifted by a normally distributed amount with a standard deviation of 0.6 weeks. The scenario sliders apply the same rules as the chart: the delay moves the *next* expected receipt, sales changes scale all receipts, and an unexpected expense falls in week 1. The outputs are the chance of going below the reserve, the chance of negative cash, the 10th, 50th and 90th percentiles per week, and the typical week of the first shortfall. The random seed is calculated from the inputs, so results can be repeated exactly.

Reminder ladder

The ladder has five steps: gentle at −7 and 0 days, formal at +7 and +15 days, legal at +30 days. Each run of the daily job sends only the latest step reached and never goes backwards, so missed early steps are skipped rather than sent late. At +15 days, Udyam-registered sellers get an estimate of MSMED Section 16 interest: three times the RBI Bank Rate, compounded monthly, starting no later than 45 days after the invoice. The legal notice includes a reference number, a 15-day deadline, the dates of earlier reminders and, for registered MSEs, the Section 18 route. A test address setting sends every reminder to the owner, and real customer emails need an explicit live switch.

Other technical decisions

- **Safe team saving.** Each save carries the workspace version it started from, and the database update succeeds only if that version is still current. On a conflict, the browser merges item by item, keeping both people's changes.
- **Security.** Passwords use scrypt. Session tokens are stored only as hashes. Cookies are HttpOnly, SameSite=Lax and Secure. Cross-site write requests are refused. Sign-in, sign-up and scanning are rate-limited. Wildcard characters in usernames are escaped. All secrets stay in server settings.
- **One serverless function.** Vercel's catch-all route files do not work outside Next.js, and this broke sign-in on an earlier deployment. One rewrite to a single function now serves every path reliably.

7 Achievements during the hackathon

Milestone	Status
Forecast, scenarios, reserve check	**Working** live and in the demo
Sign-up, sign-in, Google sign-in	**Working** live. A Google account signed in on the deployed site.
Supabase storage and shared workspaces	**Working** live, with invite and join
Invoices, paid history, MSMED interest estimate	**Working**
Invoice scanning (photo and PDF)	**Implemented** The rules are tested. Live scanning depends on Gemini capacity; on 25 Oct the newest model returned "high demand" errors, which led to the automatic model fallback.
Reminder ladder, preview, printable legal notice	**Working** preview and notice. **Needs setup** to send email: a Resend key and running `reminders.sql`.
Customer risk and deal check	**Working** . Matches the Python model on 18 of 18 test customers.
Chance of a shortfall (Monte Carlo)	**Working**
Bugs found and fixed during the hackathon	More than 20, including: login broken on Vercel, teammates overwriting each other, a data leak in demo mode after sign-out, the last edit lost on sign-out, wildcard username matching, scenario sliders not affecting the chart, a missing minus sign, and Home appearing to sign the user out.

8 Testing and validation

Suite	Result	What it covers
Browser and API regression (`tests/e2e`)	63 / 63	Sign-up and sign-in (by username and by phone), wrong password, duplicate and wildcard usernames, sessions, safe saving with a simulated teammate conflict, invites, scanning UI with simulated results, GSTIN validation, ladder statuses, risk view and deal check, demo isolation after sign-out, origin blocking, rate limits, and no page errors
Google sign-in and routing	13 / 13	Valid token, a second client ID, a token for another app (refused), an unverified email (refused), and Vercel routing with both URL forms
Scanner rules (`tests/extract.test.mjs`)	11 / 11	Indian amounts and amounts in words, ambiguous dates, payment terms, GSTIN checksum, a taxable value misread as the total, a buyer/seller swap, the second reading fixing the total, and recovery when Google fails
Reminder ladder (`tests/reminders.test.mjs`)	12 / 12	Tier order, the legal step only with approval, never going backwards, wording of each tier, preview statuses, and escaping of customer text
Risk model parity (`tests/risk-parity.mjs`)	18 / 18	The same dataset run through the original Python model and the JavaScript port: same decision for every customer, scores within 1 point, plus 33 deal checks with no errors

Sample inputs and outputs

Input	Output
Scanner reads total "2,00,000", words "Two Lakh Thirty Six Thousand", subtotal 2,00,000 + GST 36,000	Flags "looks like the taxable value, not the grand total", reads again, and returns ₹2,36,000
Date "05/10/2026"	5 October 2026, flagged as possibly the other way round
Customer with delays 12, 25, 18, 40, 33, 52, 47, 61 days, 5 of 7 promises broken, an invoice 38 days overdue	Score 68, **Advance only**. Advice: collect the overdue amount first, take 100% in advance, and call because payments are slowing down.
₹15,000 cash, ₹17,000 reserve, ₹1,80,000 receipt in week 2, sales −30%	Chance below reserve rises from 80% to 100%. The message says "already below your reserve today".

Known failures and edge cases. Gemini can return "busy" errors; the scanner then falls back to other models or asks the user to type the invoice in. Accuracy on a large set of real invoices has not been measured. The browser tests use a stand-in database and simulated Google and Gemini responses, so the live services were checked by hand.

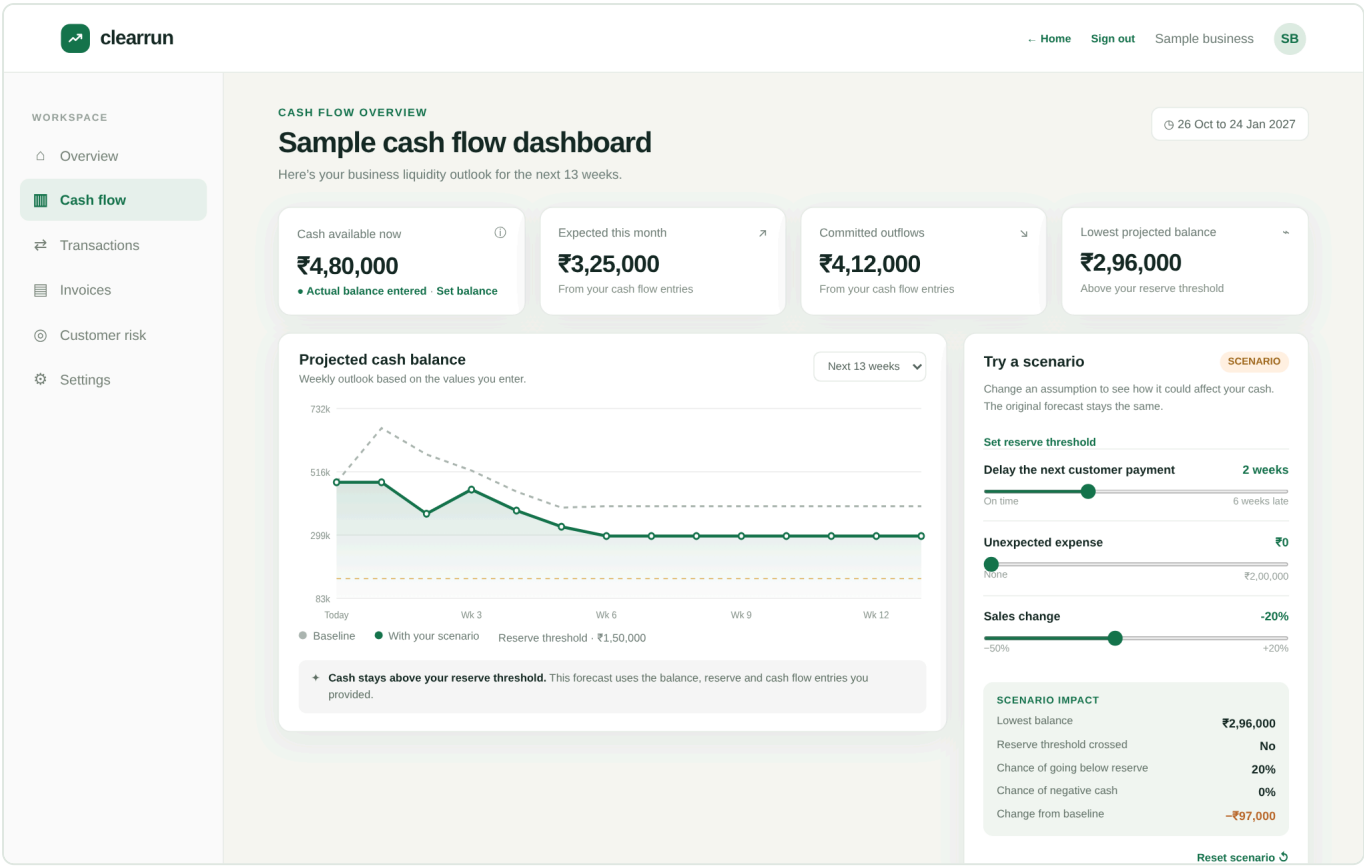
9 AI, data and external dependencies

Item	Use and attribution
Google Gemini API	Reads invoice images and PDFs at runtime (Gemini 3.8 Flash, 3.6 Flash and 3.5 Flash-Lite as fallbacks). The prompt and the JSON format it must return are in `lib/invoice-extract.mjs`. Invoice text is treated as data, never as instructions.
AI coding assistants	Claude (Anthropic) was used as an AI coding assistant during the hackathon: diagnosing deployment and login problems, merging two diverging versions of the app, porting the team's Python risk model to JavaScript and checking it against the original, adding the reminder ladder and scanner checks, writing automated tests, and drafting documentation. The team decided the features, reviewed the changes, configured and deployed the services, and tested the live app.
Data	No external dataset. The demo uses built-in sample customers and invoices, clearly labelled. The risk model was checked against the team's Python model using a generated test dataset.
Services	Vercel (hosting and cron), Supabase (Postgres), Google Identity Services, Resend (email)
Libraries	None at runtime beyond Node.js 20 built-ins. Playwright is used only for testing.

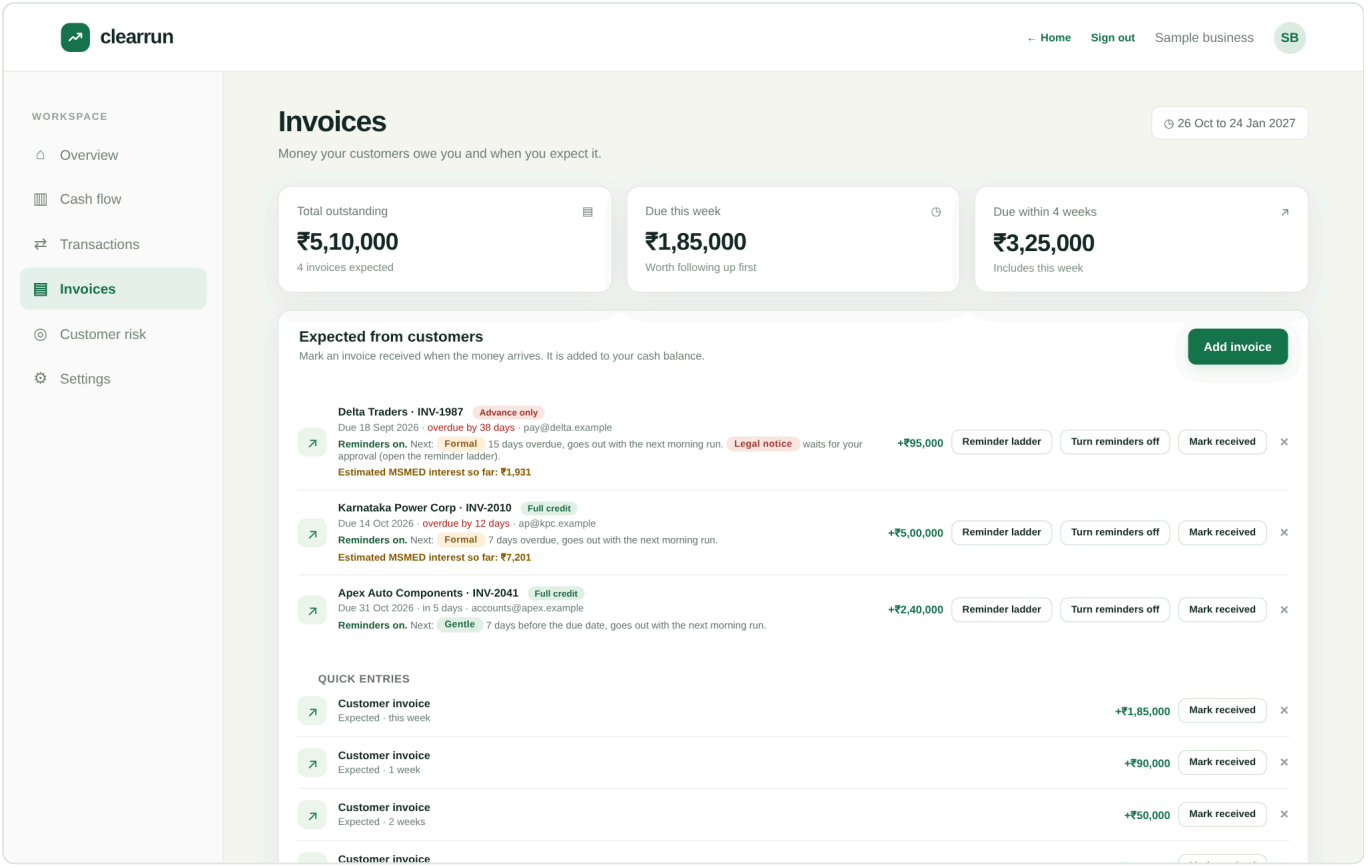
10 User interface and demonstration

Access: https://clearrun-app3.vercel.app. Choose *Explore the dashboard* for the demo, which needs no account, or sign up with a phone number or Google.

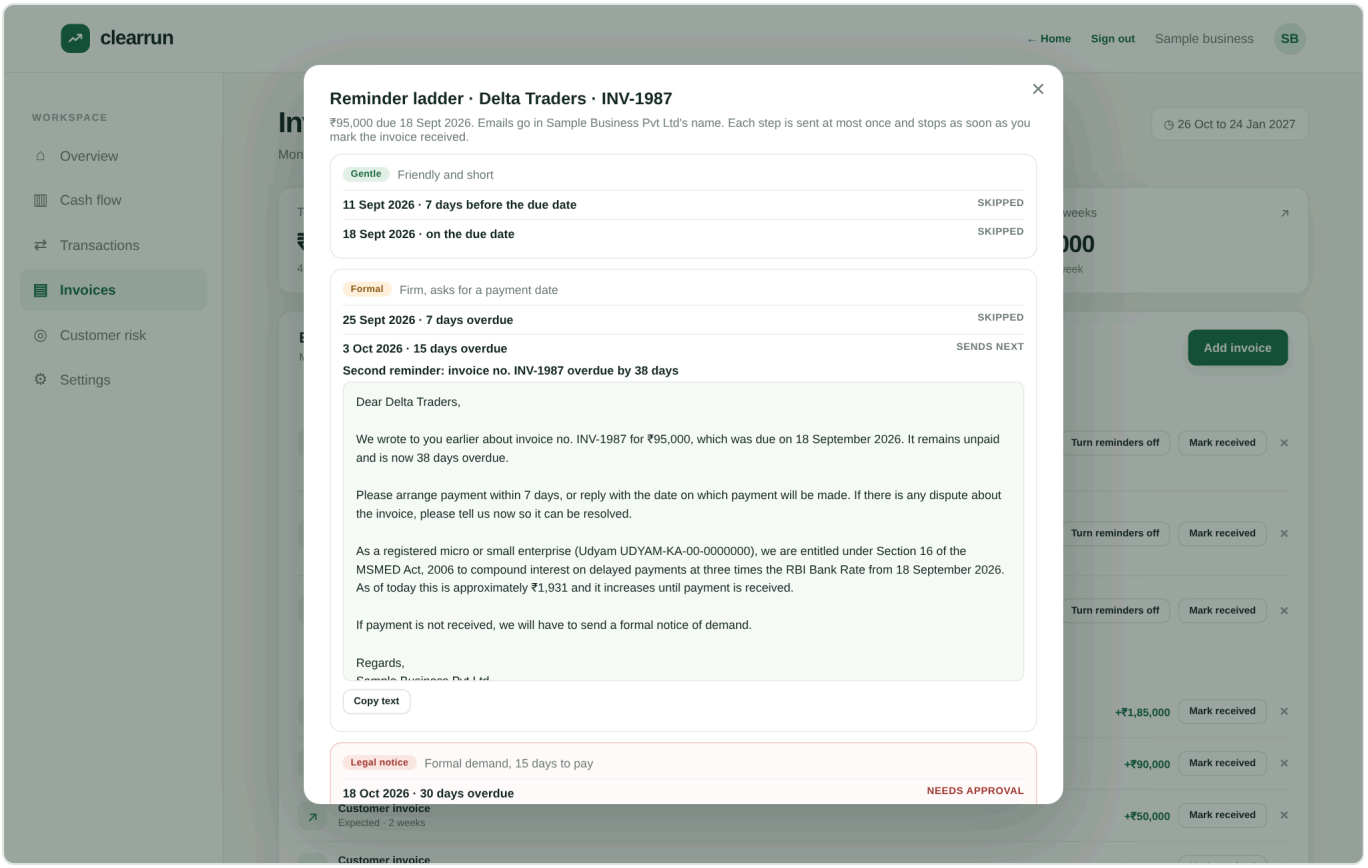
Two-minute walkthrough: (1) Look at the forecast and reserve line, then move the delay and sales sliders and watch the chance of a shortfall change. (2) In Invoices, open the *Reminder ladder* for Delta Traders, which is 38 days overdue. (3) In Customer risk, Delta Traders is "Advance only". Check a ₹1,20,000 deal and add it to the forecast. (4) Sign in and scan an invoice.



Forecast with scenario sliders and the chance of a shortfall.



Invoices with reminder status and customer risk badges.



The reminder ladder for an overdue invoice: missed steps skipped, a formal reminder next, the legal notice waiting for approval.



Sample Business Pvt Ltd
Udyam: UDYAM-KA-00-0000000

Ref: CR/INV-1987/20261026 Date: 26 October 2026

To,
Delta Traders
pay@delta.example

NOTICE OF DEMAND FOR PAYMENT

Subject: Demand for payment of ₹95,000 against invoice no. INV-1987

Dear Sir/Madam,

This notice concerns invoice no. INV-1987 dated 19 August 2026 for ₹95,000, raised by Sample Business Pvt Ltd on Delta Traders for goods and/or services supplied. Payment fell due on 18 September 2026.

Despite our earlier reminders, the amount remains unpaid and is now 38 days overdue.

You are hereby called upon to pay the outstanding amount of ₹95,000, together with interest under Section 16 of the MSMED Act, 2006 (approximately ₹1,931 as of the date of this notice), within 15 days of receiving this notice, that is on or before 10 November 2026.

Sample Business Pvt Ltd is a micro or small enterprise registered under Udyam number UDYAM-KA-00-0000000. If payment is not received by that date, we intend to refer the matter to the Micro and Small Enterprises Facilitation Council under Section 18 of the MSMED Act, 2006, and to take any other recovery action available to us, including claims for interest and costs, without further notice.

If you have already made this payment, or you dispute the amount, please contact us in writing immediately with the details.

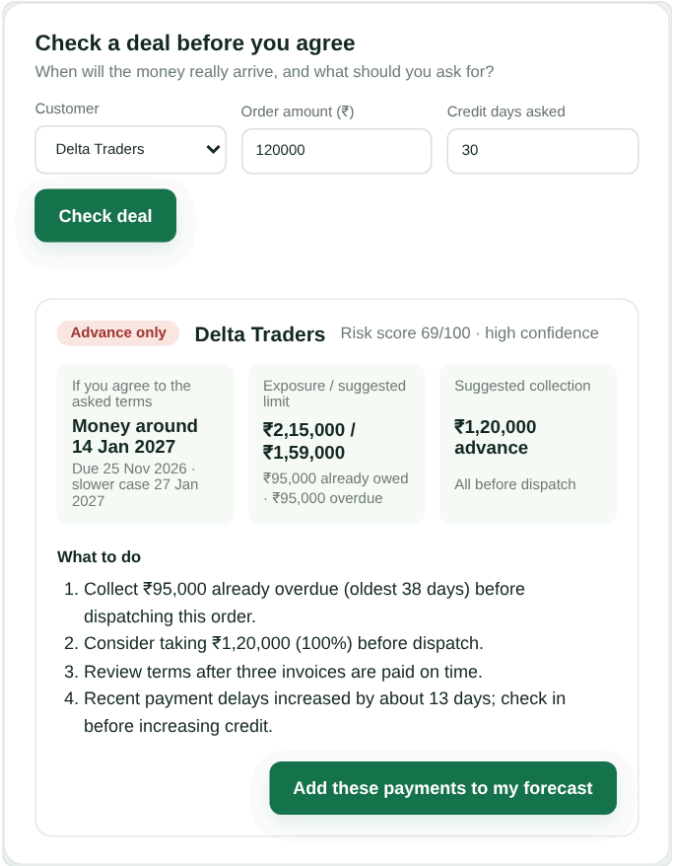
This notice is issued without prejudice to all our rights and remedies.

For and on behalf of
Sample Business Pvt Ltd

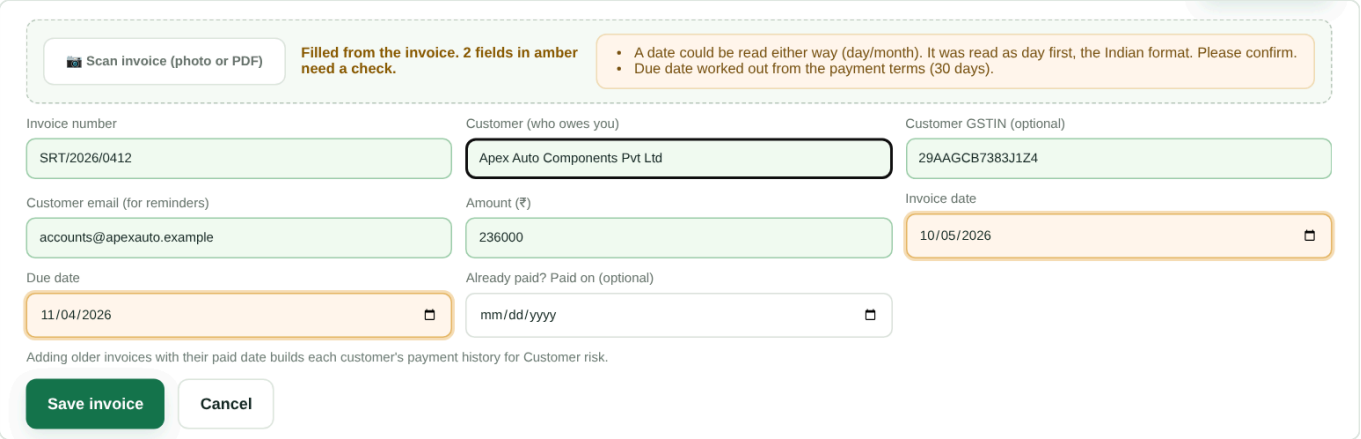
Authorised signatory

Generated by ClearRun from your invoice details. This is a template, not legal advice. Check every detail, and consider having an advocate review or issue the notice for disputes or large amounts.

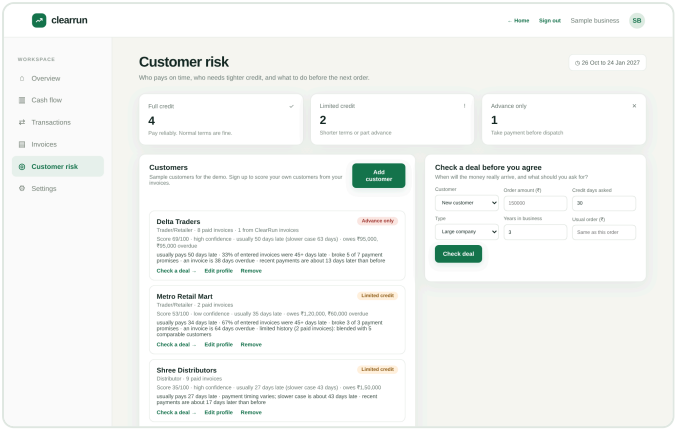
Printable legal notice (a template, not legal advice).



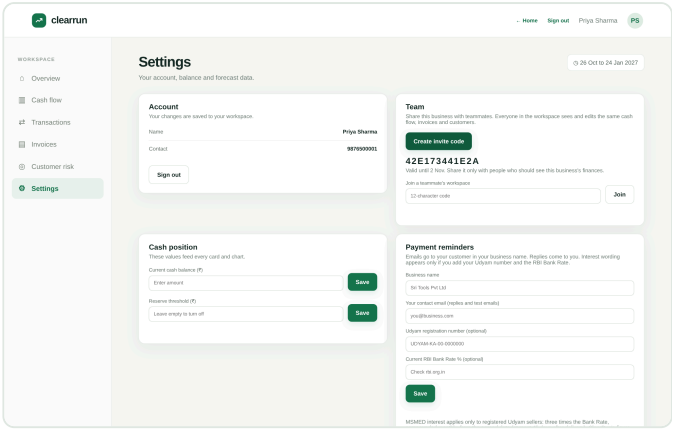
Deal check: decision, expected arrival, and suggested actions.



A scanned invoice. Fields that need a check are in amber, and the warnings are listed. The values shown come from a simulated reading.



Customer risk, with the reasons behind each score.



Settings: team invite code, cash position, and reminder details.

11 Limitations and incomplete work

Item	Honest status
Scanner accuracy	Not measured on a large set of real invoices. Handwritten and blurred bills will need more correction. The user always reviews before saving.
Reminder emails	Implemented and tested with a simulated email service. Sending from the live site needs a Resend key, a verified sending domain, and `reminders.sql`. This was not demonstrated live during the hackathon.
Simulation timing	Uses the same ±0.6-week variation for every customer; it does not yet use each customer's own payment pattern.
Risk model	Verified against the Python original, but its predictions have not been checked against real customers' later payments. Small histories give "low confidence" scores, and the app says so.
Data entry	No bank or accounting-software import. The cash balance and bills are entered by hand.
Google sign-in	The Google app is in testing mode, so only listed test users can sign in with Google until it is published. Phone and password works for everyone.
Storage design	Each workspace is one JSON document. That is fine for small businesses, but very large invoice volumes would need proper tables.

12 Real-world viability and impact

Intended users and benefits. Small B2B sellers get earlier warning of cash gaps, fewer hours chasing payments, safer credit terms, and less typing.

Feasibility and cost. It runs on free or low-cost tiers of Vercel and Supabase. There are no servers to manage, and scaling is automatic. The only paid parts would be Gemini and email at volume.

Security and privacy. Business financial data is sensitive. Measures include server-only database keys, row-level security, hashed passwords and sessions, and HttpOnly cookies. Production use would need India's DPDP Act, 2023 compliance (consent, purpose limitation, deletion on request), backups, audit logs and a security review.

Risks and legal limits. Forecasts and risk scores are estimates, not financial advice. MSMED interest figures are estimates; the user should confirm them with an accountant. The legal notice is a template, not legal advice, and should be reviewed by an advocate for disputes or large amounts. Automated customer emails must be accurate and polite, so ClearRun runs in test mode by default and needs approval before sending a legal notice.

13 Future scope

- **Auto-mitigation copilot:** when a shortfall is predicted, rank fixes (chase this invoice, ask that customer for an advance, move a flexible bill) by simulated effect, and apply one with a click. The estimate is about 4 days for the rule-based version.
- Use each customer's delay pattern from the risk model in the simulation.
- Bank balances through India's Account Aggregator framework (with consent), and import from Tally, Zoho Books and GST e-invoices.
- WhatsApp reminders, and measuring scanner accuracy on a labelled set of real invoices.
- Proper invoice tables, audit history, account recovery, monitoring and backups for production.

14 Setup and reproducibility

1. Install Node.js 20+. Clone the repository: https://github.com/rohinii-dev/clearrun
2. In Supabase → SQL Editor, run `supabase/schema.sql` and then `supabase/reminders.sql`.
3. Copy `.env.example` to `.env` and fill in `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` and `GOOGLE_CLIENT_ID`. Optional: `GEMINI_API_KEY`, and `RESEND_API_KEY` / `RESEND_FROM` / `CRON_SECRET` / `REMINDER_TEST_TO`.
4. Run `npm run dev` and open http://localhost:5500. Check http://localhost:5500/api/health.
5. Tests: `npm test` (unit tests) and `npm run test:e2e` (needs Playwright; uses a stand-in database, so no keys are needed).
6. Deploy: `npx vercel`, add the same settings with `npx vercel env add`, then `npx vercel --prod`.

Demo data: choose "Explore the dashboard" for sample cash flows, invoices and customers. No secrets, API keys or real customer data are included in the repository or this report.

15 Team contributions and references

Member	Contribution
Srivatsan Reddy Moola	Frontend and UI: dashboard screens, layout and design. Research, testing and report: problem research, testing, documentation and pitch.
Smrithi Pramanik	Frontend and UI: dashboard screens, layout and design. Research, testing and report: problem research, testing, documentation and pitch.
Anushree D	Backend and deployment: server API, Supabase database, Vercel deployment, Google sign-in. Models: customer risk model, cash forecast simulation and invoice scanner.
Rohini U Gachinmath	Backend and deployment: server API, Supabase database, Vercel deployment, Google sign-in. Models: customer risk model, cash forecast simulation and invoice scanner.

References

- Micro, Small and Medium Enterprises Development (MSMED) Act, 2006: Sections 15–18 (payment timelines, interest at three times the bank rate, MSE Facilitation Council).
- GSTIN format and check-character scheme (15-character GST identification number).
- Google Gemini API model list: ai.google.dev/models
- Vercel Functions, rewrites and Cron documentation; Supabase (PostgREST) documentation; Google Identity Services documentation.
- Digital Personal Data Protection Act, 2023 (India).

Companion documents in the repository: `docs/ClearRun-Architecture.pdf`, `docs/ClearRun-Models.pdf`, `docs/ClearRun-Problem-Statement-F03.pdf`.